Play It Again

Sometimes you may be able to correct the cause of an exception. In those cases, you can use the retry statement within a rescue clause to repeat the entire begin/end block. Clearly, tremendous scope exists for infinite loops here, so this is a feature to use with caution (and with a finger resting lightly on the interrupt key).

As an example of code that retries on exceptions, take a look at the following, which is very simplified code that you might find making a network connection.

```
attempts = 0
begin
  attempts += 1
  @connection = @remote_server.read_data

rescue TimeOutError
  if @remote_server && attempts < 10 then
    sleep(attempts ** 2)
    retry
  else
    raise
  end
end
```

This code tries to read data from remote_server. If the code returns a TimeOutError and if the remote_server exists, the code sleeps for a while, then tries again. It keeps track of the number of attempts, lengthening the time out, until eventually if the number of attempts gets too high, it stops trying to connect and just raises the error.

Raising Exceptions

So far, we've been on the defensive, handling exceptions raised by others. It's time to turn the tables and go on the offensive. It's time to raise some… exceptions.

You can raise exceptions in your code with the raise method (or its somewhat judgmental and less commonly used synonym, fail):

```
raise
raise "bad mp3 encoding"
raise InterfaceException, "Keyboard failure"
```

The first form simply reraises the current exception (or raises a RuntimeError if there is no current exception). This is used in exception handlers that intercept an exception before passing it on.

The second form creates a new RuntimeError exception, setting its message to the given string. This exception is then raised up the call stack.

The third form uses the first argument to create an exception and then sets the associated message to the second argument. Typically the first argument will be either the name of a class in the Exception hierarchy or a reference to an instance of one of these classes—if the argument is a class name, then Ruby will create an instance using a call to new with no arguments. Technically, this argument can be any object that responds to the message exception by returning an object such that object.kind_of?(Exception) is true.

Here are some typical examples of raise in action: